**WEEK 10 – ADVANCED SQL QUERY SYSTEM**

**AIM**

To implement advanced SQL queries using subqueries and nested queries in Oracle SQL to analyze travel package prices, customer purchases, booking details, and business performance.

**INTRODUCTION**

A subquery is an SQL query written inside another SQL query. It helps retrieve data based on the result of another query.

Subqueries are used to perform complex operations such as finding the highest value, calculating average prices, identifying valuable customers, and generating business reports.

The types of subqueries covered are:

- Single-row subquery
- Multi-row subquery
- Nested subquery

**IMPLEMENT SUBQUERIES AND NESTED QUERIES**

**Find Travel Packages Above Average Price**

**Description:** Display travel packages whose price is greater than the average price of all packages.

```
SQL> SELECT PACKAGE_ID,
  2         PACKAGE_NAME,
  3         CATEGORY_ID,
  4         PRICE
  5  FROM TRAVELPACKAGE
  6  WHERE PRICE > (
  7      SELECT AVG(PRICE)
  8      FROM TRAVELPACKAGE
  9  );

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
CATEGORY_ID      PRICE
----------- ----------
       103
Dubai Holiday Package
          2      55000

       104
Singapore Tour Package
          2      65000

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
CATEGORY_ID      PRICE
----------- ----------

       107
Goa Honeymoon Package
          4      30000

       108
Kashmir Honeymoon Package
```

```
Priya Sharma                              C002                                1
         36000


8 rows selected.
```

**Identify Expensive Packages in Each Category**

**Description:** Display the highest-priced package in each category.

```
SQL> SELECT PACKAGE_ID, PACKAGE_NAME, CATEGORY_ID, PRICE
  2  FROM TRAVELPACKAGE P
  3  WHERE PRICE = (
  4      SELECT MAX(PRICE)
  5      FROM TRAVELPACKAGE
  6      WHERE CATEGORY_ID = P.CATEGORY_ID
  7  );

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
CATEGORY_ID      PRICE
----------- ----------
       101
Ooty Holiday Package
          1      13000

       104
Singapore Tour Package
          2      65000

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
CATEGORY_ID      PRICE
----------- ----------

       105
Manali Adventure Trip
          3      25000

       108
Kashmir Honeymoon Package
```

```
PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
CATEGORY_ID      PRICE
----------- ----------
          4      45000

       110
Rajasthan Family Tour
          5      28000

       112

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
CATEGORY_ID      PRICE
----------- ----------
Varanasi Spiritual Tour
          6      14000


6 rows selected.
```

**Packages Above Selected Category Average**

**Description:** Display packages in Category 1 whose price is greater than that category's average price.

```
SQL> SELECT PACKAGE_ID, PACKAGE_NAME, CATEGORY_ID, PRICE
  2  FROM TRAVELPACKAGE
  3  WHERE CATEGORY_ID = 1
  4  AND PRICE > (
  5      SELECT AVG(PRICE)
  6      FROM TRAVELPACKAGE
  7      WHERE CATEGORY_ID = 1
  8  );

no rows selected
```

**IDENTIFY CUSTOMERS WITH MAXIMUM PURCHASES**

**Customer Who Spent the Highest Amount**

```
SQL> SELECT CUSTOMER_ID, SUM(TOTAL_AMOUNT) AS TOTAL_SPENT
  2  FROM BOOKING
  3  GROUP BY CUSTOMER_ID
  4  HAVING SUM(TOTAL_AMOUNT) = (
  5      SELECT MAX(TOTAL_SPENT)
  6      FROM (
  7          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENT
  8          FROM BOOKING
  9          GROUP BY CUSTOMER_ID
 10      )
 11  );

CUSTOMER_ID          TOTAL_SPENT
-------------------- -----------
C004                       65000
```

**Customers with Maximum Number of Orders**

```
SQL> SELECT CUSTOMER_ID, COUNT(BOOKING_ID) AS TOTAL_ORDERS
  2  FROM BOOKING
  3  GROUP BY CUSTOMER_ID
  4  HAVING COUNT(BOOKING_ID) = (
  5      SELECT MAX(TOTAL_ORDERS)
  6      FROM (
  7          SELECT COUNT(BOOKING_ID) AS TOTAL_ORDERS
  8          FROM BOOKING
  9          GROUP BY CUSTOMER_ID
 10      )
 11  );

CUSTOMER_ID          TOTAL_ORDERS
-------------------- ------------
C004                            1
C006                            1
C008                            1
C007                            1
C003                            1
C005                            1
C001                            1
C002                            1

8 rows selected.
```

**Customers Whose Spending Is Above Average**

```
SQL> SELECT CUSTOMER_ID, SUM(TOTAL_AMOUNT) AS TOTAL_SPENT
  2  FROM BOOKING
  3  GROUP BY CUSTOMER_ID
  4  HAVING SUM(TOTAL_AMOUNT) > (
  5      SELECT AVG(TOTAL_SPENT)
  6      FROM (
  7          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENT
  8          FROM BOOKING
  9          GROUP BY CUSTOMER_ID
 10      )
 11  );

CUSTOMER_ID          TOTAL_SPENT
-------------------- -----------
C004                       65000
C008                       55000
C007                       45000
C003                       55000
```

**Top 5 Valuable Customers**

```
SQL> SELECT *
  2  FROM (
  3      SELECT CUSTOMER_ID,
  4             SUM(TOTAL_AMOUNT) AS TOTAL_SPENT
  5      FROM BOOKING
  6      GROUP BY CUSTOMER_ID
  7      ORDER BY TOTAL_SPENT DESC
  8  )
  9  WHERE ROWNUM <= 5;

CUSTOMER_ID          TOTAL_SPENT
-------------------- -----------
C004                       65000
C008                       55000
C003                       55000
C007                       45000
C002                       36000
```

**GENERATE COMPLEX BUSINESS QUERIES**

**Query 1: Best-Selling Travel Package**

**Description:** Display the highest-priced travel package.

```
SQL> SELECT PACKAGE_ID, PACKAGE_NAME, PRICE
  2  FROM TRAVELPACKAGE
  3  WHERE PRICE = (
  4      SELECT MAX(PRICE)
  5      FROM TRAVELPACKAGE
  6  );

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
     PRICE
----------
       104
Singapore Tour Package
     65000
```

**Query 2: High-Value Customers**

**Description:** Display customers whose total purchase amount is greater than the average booking amount.

```
SQL> SELECT CUSTOMER_ID,
  2         SUM(TOTAL_AMOUNT) AS TOTAL_SPENT
  3  FROM BOOKING
  4  GROUP BY CUSTOMER_ID
  5  HAVING SUM(TOTAL_AMOUNT) > (
  6      SELECT AVG(TOTAL_AMOUNT)
  7      FROM BOOKING
  8  );

CUSTOMER_ID          TOTAL_SPENT
-------------------- -----------
C004                       65000
C008                       55000
C007                       45000
C003                       55000
```

**Query 3: Category Performance Analysis**

**Category with the Highest Average Package Price**

```
SQL> SELECT CATEGORY_ID, AVG(PRICE) AS AVG_PRICE
  2  FROM TRAVELPACKAGE
  3  GROUP BY CATEGORY_ID
  4  HAVING AVG(PRICE) = (
  5      SELECT MAX(AVG_PRICE)
  6      FROM (
  7          SELECT AVG(PRICE) AS AVG_PRICE
  8          FROM TRAVELPACKAGE
  9          GROUP BY CATEGORY_ID
 10      )
 11  );

CATEGORY_ID  AVG_PRICE
----------- ----------
          2      60000
```

**Number of Packages in Each Category**

```
SQL> SELECT CATEGORY_ID, COUNT(PACKAGE_ID) AS TOTAL_PACKAGES
  2  FROM TRAVELPACKAGE
  3  GROUP BY CATEGORY_ID;

CATEGORY_ID TOTAL_PACKAGES
----------- --------------
          1              1
          6              2
          2              2
          4              2
          5              2
          3              2

6 rows selected.
```

**Average Package Price by Category**

```
SQL> SELECT CATEGORY_ID, AVG(PRICE) AS AVERAGE_PRICE
  2  FROM TRAVELPACKAGE
  3  GROUP BY CATEGORY_ID;

CATEGORY_ID AVERAGE_PRICE
----------- -------------
          1         13000
          6         11500
          2         60000
          4         37500
          5         21500
          3         23500

6 rows selected.
```

**Query 4: Customer Purchase History**

**Description:** Display customer name, number of bookings, and total purchase amount without using JOIN.

```
SQL> SELECT
  2      (SELECT FIRST_NAME || ' ' || LAST_NAME
  3       FROM CUSTOMER C
  4       WHERE C.CUSTOMER_ID = B.CUSTOMER_ID) AS CUSTOMER_NAME,
  5      CUSTOMER_ID,
  6      COUNT(BOOKING_ID) AS TOTAL_ORDERS,
  7      SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  8  FROM BOOKING B
  9  GROUP BY CUSTOMER_ID;

CUSTOMER_NAME                          CUSTOMER_ID          TOTAL_ORDERS
-------------------------------------- -------------------- ------------
TOTAL_PURCHASE
-------------
Divya Suresh                           C004                            1
         65000

Sneha Ravi                             C006                            1
         30000

Ananya Krishnan                        C008                            1
         55000


CUSTOMER_NAME                          CUSTOMER_ID          TOTAL_ORDERS
-------------------------------------- -------------------- ------------
TOTAL_PURCHASE
-------------
Vishnu Kumar                           C007                            1
         45000

Rahul Raj                              C003                            1
         55000

Karthik Mohan                          C005                            1
         25000


CUSTOMER_NAME                          CUSTOMER_ID          TOTAL_ORDERS
-------------------------------------- -------------------- ------------
TOTAL_PURCHASE
-------------
Arun Kumar                             C001                            1
         24000

Priya Sharma                           C002                            1
         36000


8 rows selected.
```

**PREPARE ADVANCED SQL REPORTS**

**Report 1: Premium Package Report**

**Description:** Display travel packages whose price is above the average package price.

```
SQL> SELECT PACKAGE_ID,
  2         PACKAGE_NAME,
  3         CATEGORY_ID,
  4         PRICE,
  5         AVAILABLE_SLOTS
  6  FROM TRAVELPACKAGE
  7  WHERE PRICE > (
  8      SELECT AVG(PRICE)
  9      FROM TRAVELPACKAGE
 10  );

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
CATEGORY_ID      PRICE AVAILABLE_SLOTS
----------- ---------- ---------------
       103
Dubai Holiday Package
          2      55000              10

       104
Singapore Tour Package
          2      65000              12

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
CATEGORY_ID      PRICE AVAILABLE_SLOTS
----------- ---------- ---------------

       107
Goa Honeymoon Package
          4      30000              10

       108
Kashmir Honeymoon Package
```

```
PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
CATEGORY_ID      PRICE AVAILABLE_SLOTS
----------- ---------- ---------------
          4      45000               8
```

**Report 2: Customer Value Report**

**Description:** Display customer name, total orders, and total spending.

```
SQL> SELECT
  2      (SELECT FIRST_NAME || ' ' || LAST_NAME
  3       FROM CUSTOMER C
  4       WHERE C.CUSTOMER_ID = B.CUSTOMER_ID) AS CUSTOMER_NAME,
  5      CUSTOMER_ID,
  6      COUNT(BOOKING_ID) AS TOTAL_ORDERS,
  7      SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  8  FROM BOOKING B
  9  GROUP BY CUSTOMER_ID
 10  ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_NAME                          CUSTOMER_ID          TOTAL_ORDERS
-------------------------------------- -------------------- ------------
TOTAL_SPENDING
--------------
Divya Suresh                           C004                            1
         65000

Rahul Raj                              C003                            1
         55000

Ananya Krishnan                        C008                            1
         55000


CUSTOMER_NAME                          CUSTOMER_ID          TOTAL_ORDERS
-------------------------------------- -------------------- ------------
TOTAL_SPENDING
--------------
Vishnu Kumar                           C007                            1
         45000

Priya Sharma                           C002                            1
         36000

Sneha Ravi                             C006                            1
         30000
```

```
CUSTOMER_NAME                          CUSTOMER_ID          TOTAL_ORDERS
-------------------------------------- -------------------- ------------
TOTAL_SPENDING
--------------
Karthik Mohan                          C005                            1
         25000

Arun Kumar                             C001                            1
         24000


8 rows selected.
```

**Report 3: Sales Performance Report**

**Booking-Wise Revenue**

```
SQL> SELECT BOOKING_ID, CUSTOMER_ID, TOTAL_AMOUNT
  2  FROM BOOKING
  3  ORDER BY TOTAL_AMOUNT DESC;

BOOKING_ID CUSTOMER_ID          TOTAL_AMOUNT
---------- -------------------- ------------
B4         C004                        65000
B3         C003                        55000
B8         C008                        55000
B7         C007                        45000
B2         C002                        36000
B6         C006                        30000
B5         C005                        25000
B1         C001                        24000

8 rows selected.
```

**Category-Wise Package Price Analysis**

```
SQL> SELECT CATEGORY_ID,
  2         COUNT(PACKAGE_ID) AS TOTAL_PACKAGES,
  3         SUM(PRICE) AS TOTAL_PACKAGE_PRICE,
  4         AVG(PRICE) AS AVERAGE_PRICE
  5  FROM TRAVELPACKAGE
  6  GROUP BY CATEGORY_ID;

CATEGORY_ID TOTAL_PACKAGES TOTAL_PACKAGE_PRICE AVERAGE_PRICE
----------- -------------- -------------------- -------------
          1              1                13000         13000
          6              2                23000         11500
          2              2               120000         60000
          4              2                75000         37500
          5              2                43000         21500
          3              2                47000         23500

6 rows selected.
```

**Top 5 Expensive Packages**

```
SQL> SELECT *
  2  FROM (
  3      SELECT PACKAGE_ID, PACKAGE_NAME, PRICE
  4      FROM TRAVELPACKAGE
  5      ORDER BY PRICE DESC
  6  )
  7  WHERE ROWNUM <= 5;

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
     PRICE
----------
       104
Singapore Tour Package
     65000

       103
Dubai Holiday Package
     55000

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
     PRICE
----------

       108
Kashmir Honeymoon Package
     45000

       107
Goa Honeymoon Package
```

```
PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
     PRICE
----------
     30000

       110
Rajasthan Family Tour
     28000
```

**Report 4: Business Decision Report**

**High-Priced Packages**

```
SQL> SELECT PACKAGE_ID, PACKAGE_NAME, PRICE
  2  FROM TRAVELPACKAGE
  3  WHERE PRICE = (
  4      SELECT MAX(PRICE)
  5      FROM TRAVELPACKAGE
  6  );

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
----
     PRICE
----------
       104
Singapore Tour Package
     65000
```

**Low-Priced Packages**

```
SQL> SELECT PACKAGE_ID, PACKAGE_NAME, PRICE
  2  FROM TRAVELPACKAGE
  3  WHERE PRICE = (
  4      SELECT MIN(PRICE)
  5      FROM TRAVELPACKAGE
  6  );

PACKAGE_ID
----------
PACKAGE_NAME
--------------------------------------------------------------------------------
---
     PRICE
----------
       111
Tirupati Pilgrimage Tour
      9000
```

**High-Value Customers**

```
SQL> SELECT CUSTOMER_ID,
  2         SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  3  FROM BOOKING
  4  GROUP BY CUSTOMER_ID
  5  HAVING SUM(TOTAL_AMOUNT) = (
  6      SELECT MAX(TOTAL_SPENDING)
  7      FROM (
  8          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  9          FROM BOOKING
 10          GROUP BY CUSTOMER_ID
 11      )
 12  );

CUSTOMER_ID          TOTAL_SPENDING
-------------------- --------------
C004                          65000
```

**Revenue Trends by Booking Date**

```
SQL> SELECT BOOKING_DATE,
  2         COUNT(BOOKING_ID) AS TOTAL_BOOKINGS,
  3         SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE
  4  FROM BOOKING
  5  GROUP BY BOOKING_DATE
  6  ORDER BY BOOKING_DATE;

BOOKING_D TOTAL_BOOKINGS TOTAL_REVENUE
--------- -------------- -------------
10-SEP-26              1         24000
11-SEP-26              1         36000
12-SEP-26              1         55000
13-SEP-26              1         65000
14-SEP-26              1         25000
15-SEP-26              1         30000
16-SEP-26              1         45000
17-SEP-26              1         55000

8 rows selected.
```

**RESULT**

Advanced SQL queries using subqueries and nested queries were implemented successfully in Oracle SQL. Customer spending, booking counts, package prices, category statistics, and business reports were analyzed.

**CONCLUSION**

The Advanced SQL Query System was successfully completed. Subqueries and nested queries helped retrieve complex information from the Travel Booking Management System and generate useful reports for business analysis.